



Penetration Testing

Gala (Dex)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	44
Our Methodology	45
Disclaimers	47
About Hashlock	48

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Gala team partnered with Hashlock to conduct a security Penetration Test of their Gala Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Gala is building a decentralized entertainment ecosystem that spans gaming, music, film, and blockchain technology. Their platform, GalaChain, is an L1 blockchain optimized for entertainment, offering fast transaction speeds, scalability, and energy efficiency. Gala provides developers with tools like Gala Creators and GalaChain Connect to create and deploy Web3 applications seamlessly. The ecosystem supports various projects, including games like Champions Arena and Spider Tanks, Gala Music for blockchain-based music distribution, and Gala Film for decentralized cinema experiences. Gala's mission is to empower creators and players by providing them with real ownership and control over their digital assets.

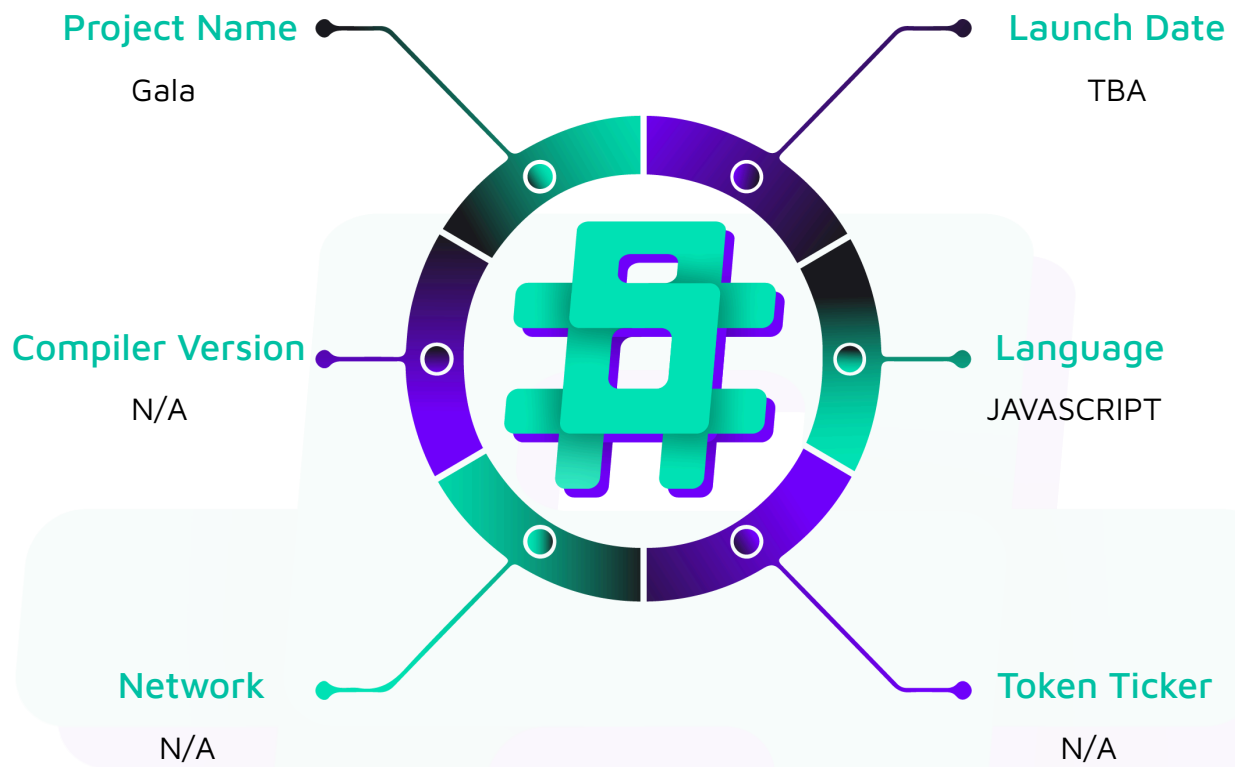
Project Name: Gala

Project Type: dApp, Token

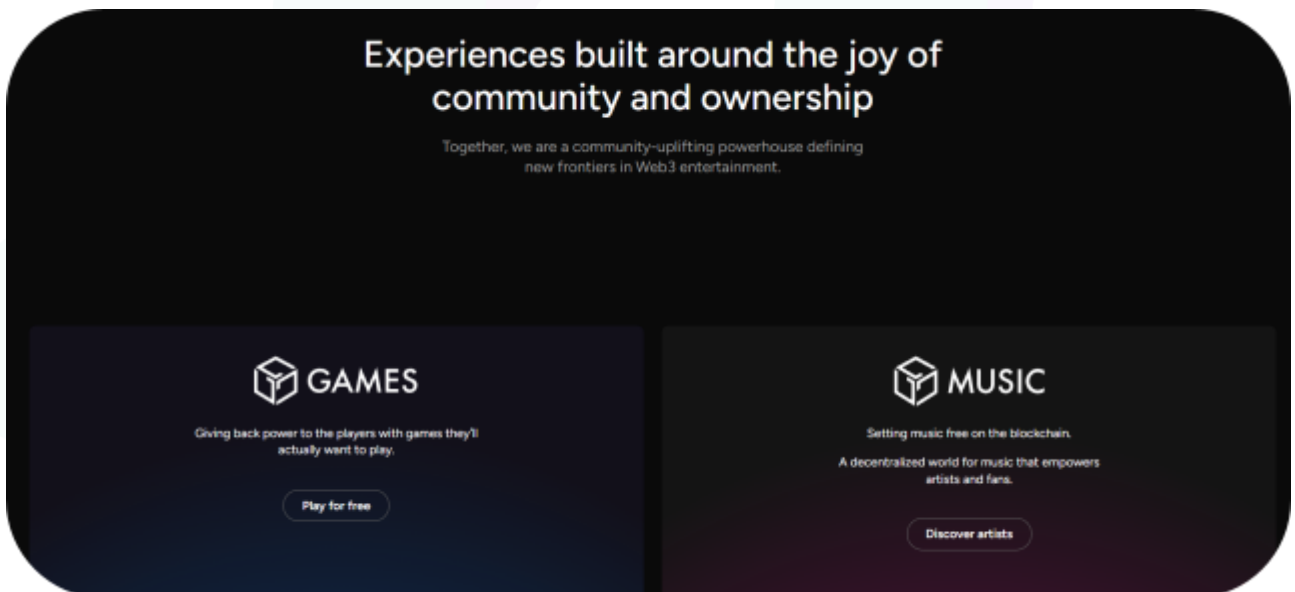
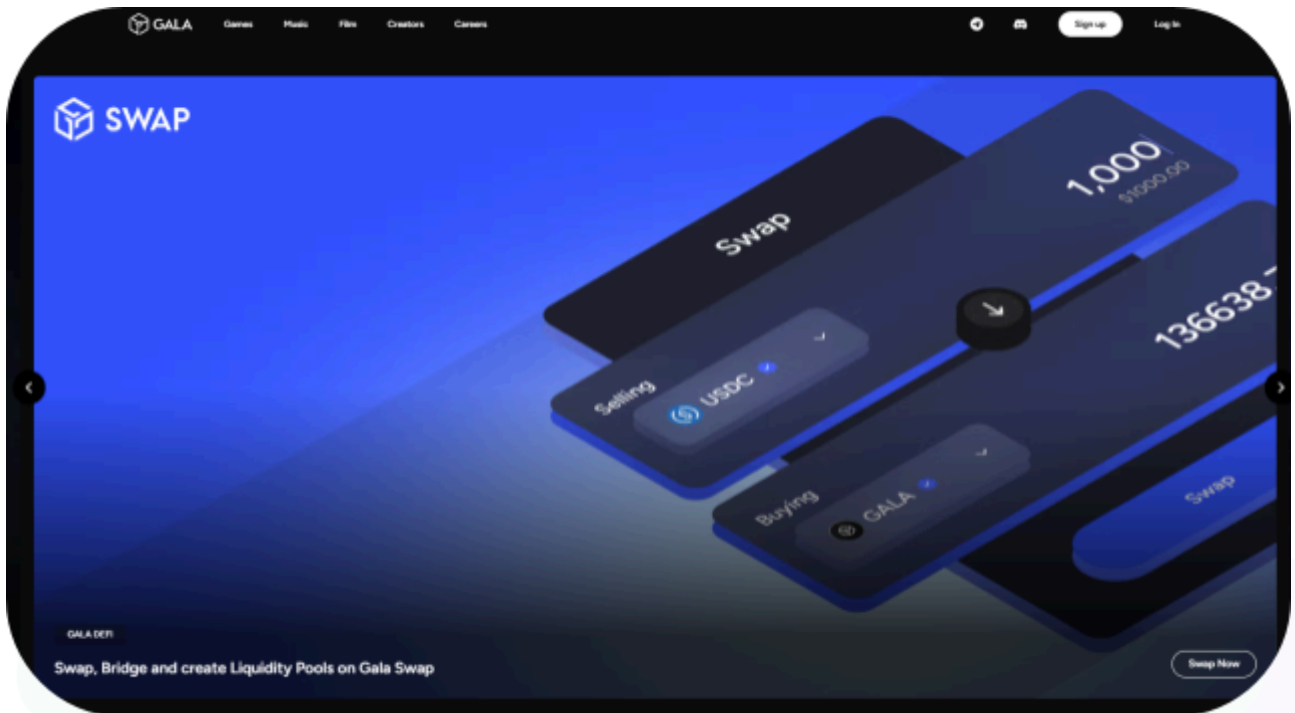
Website: gala.com

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Gala project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Gala Code Base
Platform	Web apps & APIs
Penetration Test Date	June, 2025
Component 1	Historical Oracle lambda (lambda)
Repository	https://gitlab.com/gala-games/defi/dex/lamda
Audited Commit Hash	86f7f638885c78b5eeadb70d97a90838db570900
Component 2	Frontend (defi-frontend)
Repository	https://gitlab.com/gala-games/defi/dex/defi-frontend
Audited Commit Hash	8344a1379e29c6463eaa03efffb0763a2c5d8cf
Component 3	DEX-Backend (defi-backend)
Repository	https://gitlab.com/gala-games/defi/dex/defi-backend
Audited Commit Hash	2d4e7be8f8d47bd22b182efa282e6925e481ae4e
Component 4	Bundling-Backend
Repository	https://gitlab.com/gala-games/defi/bundling/backend
Commit Hash	814f97af2f4863a8e292adef77f431126172554
Component 5	DEX chaincode review (dex)
Repository	https://github.com/GalaChain/dex
Commit Hash	f696269088a1fc73e30153dadfb9e789a026d628



Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed, and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

9 Medium severity vulnerabilities

11 Low severity vulnerabilities

5 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] defi-backend - Unrestricted faucet requests and potential deployment on mainnet

Description

The `/transfer-faucets` endpoint in `UserController` allows anyone to mint test tokens from the admin wallet without any authentication or rate-limiting.

Vulnerability Details

In `src/api/user/user.controller.ts`, the `transferFunds` handler invokes `userService.transferFaucets` solely based on the request body and immediately returns success:

```
@Post('/transfer-faucets')

public async transferFunds(@Body() data: TransferFaucetsDto, @Response() res: any) {

    const transferFaucets = await this.userService.transferFaucets(data.address);

    return res.status(transferFaucets.status).send({ message: transferFaucets.message
});

}
```

There is no check of caller identity or limits on how often a given address can request faucets, allowing users to request many faucet tokens.

Impact

An unauthenticated attacker can repeatedly call this endpoint to drain the admin's faucet balance, potentially disrupting all downstream testing and service functionality.

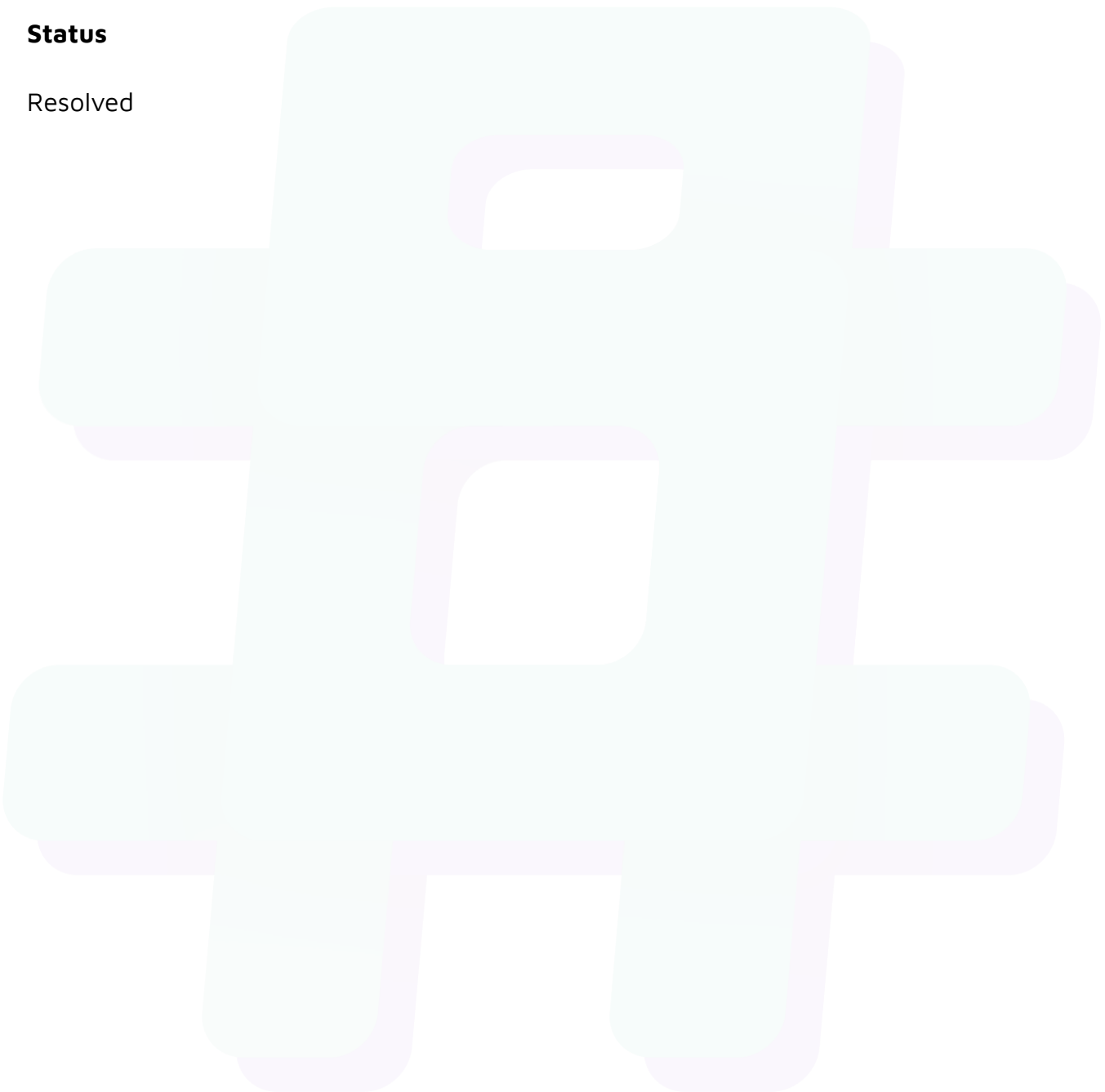
Additionally, if this deployment is for the mainnet environment, real-world funds will be sent instead of worthless ones.

Recommendation

- Require callers to authenticate (for example, via JWT) and enforce per-address rate limits or CAPTCHAs on fountain endpoints.
- Implement validation to ensure this endpoint cannot be called when it is deployed on the mainnet, thereby preventing the distribution of real-world funds.

Status

Resolved



[H-02] defi-backend - Unauthenticated Redis cache manipulation via cache-test

Description

The /v1/trade/cache-test endpoint allows anyone to overwrite or read arbitrary Redis keys without authentication.

Vulnerability Details

In src/trade/trade.service.ts:

```
async cacheTest(params: any) {  
  const key = `TESTING_${params.token}`;  
  
  if (params.type == 'set') {  
    await this.cacheService.set(key, JSON.stringify(params.data),  
      Number(params.time));  
  
    return '1';  
  } else {  
    return this.cacheService.get(key);  
  }  
}
```

No access control is applied to reads or writes.

Example access:

```
GET /v1/trade/cache-test?token=foo2&type=set&data=he<s>aaaa</s>llo_world&time=60  
HTTP/2  
  
Host: dex-backend-test3.defi.gala.com  
  
Accept: */*  
  
GET /v1/trade/cache-test?token=foo2&type=get HTTP/2
```

Host: dex-backend-test3.defi.gala.com

Accept: */*

Response:

HTTP/2 200 OK

Date: Sun, 22 Jun 2025 15:17:22 GMT

Content-Type: text/html; charset=utf-8

Content-Length: 24

Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests

Cross-Origin-Opener-Policy: same-origin

Cross-Origin-Resource-Policy: same-origin

Origin-Agent-Cluster: ?1

Referrer-Policy: same-origin

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

X-Content-Type-Options: nosniff

X-Dns-Prefetch-Control: off

X-Download-Options: noopen

X-Frame-Options: SAMEORIGIN

X-Permitted-Cross-Domain-Policies: none

X-Xss-Protection: 0

Access-Control-Allow-Origin: *

Access-Control-Allow-Credentials: true

Etag: W/"18-LcmfuY2VQ+35Nj01qCLUtfozCgE"



```
Vary: Accept-Encoding
```

```
"he<s>aaaa</s>llo_world"
```

Impact

Attackers can poison cache entries used elsewhere in the application or exfiltrate cached data. While the CSP blocks potential XSS attacks, an HTML injection is still technically possible as an additional impact.

While users can overwrite TESTING[...] keys, they cannot overwrite legitimate application keys. However, each call creates a new Redis key TESTING_<token>. With enough unique <token> values, users can fill up the Redis memory, causing either the eviction of other entries or an OOM kill or denial-of-service.

Recommendation

Remove or disable this endpoint in production, or restrict it behind strong authentication and authorization.

Status

Resolved

[H-03] oracle-lambda - price_snapshots table is not created**Description**

In line 115 of the `index.js` file, the SQL statement inserts an update to the `price_snapshots` table. However, this table had not been created previously, causing the SQL insert statement to fail.

Impact

Token prices will not be stored.

Recommendation

Consider creating the `price_snapshots` table before fetching token prices.

Status

Resolved

[H-04] oracle-lambda - Incorrect update of fee24h and tvl

Description

In line 276 of the `index.js` file, the SQL `INSERT` statement is not in the correct order. The logic tries to update the `pool_details` table with the order as follows:

```
pool_hash, token0, token0_image, token1, token1_image, fee, fee24h, tvl, volume1d,
volume30d
```

However, when the retrieved parameters are supplied to the query in line 290, it passes as follows:

```
pool_hash, token0, token0_image, token1, token1_image, fee, tvl, fee24h, volume1d,
volume30d
```

Note that the `fee_1d_usd` variable represents `fee24h`, while the `tvl_usd` variable represents `tvl`. This will cause the order for `fee24h` and `tvl` to be incorrect.

Impact

The `tvl_usd` value will be incorrectly stored into `fee24h`, and the `fee24h` value will be incorrectly stored into `tvl`.

Recommendation

Consider switching the orderings.

Status

Resolved

Medium

[M-01] oracle-lambda - Insecure secrets handling

Description

The Lambda function in line 317 of `index.js` logs sensitive AWS Secrets Manager credentials directly to CloudWatch logs, potentially exposing database credentials, API keys, and authentication tokens to anyone with access to the logs.

Vulnerability Details

After retrieving secrets from AWS Secrets Manager, the code explicitly logs the entire credentials object, including all sensitive values:

```
const creds = await getSecrets();  
  
console.log('Creds are: ', creds)
```

The `getSecrets()` function returns parsed JSON containing `MYSQL_HOST`, `MYSQL_USERNAME`, `MYSQL_PASSWORD`, `MYSQL_DATABASE`, `API_USERNAME`, `API_PASSWORD`, and other sensitive configuration values. These credentials are then permanently stored in CloudWatch logs, where they can be accessed by anyone with log reading permissions, including potential insider threats or compromised accounts.

Impact

Complete exposure of database credentials and API authentication tokens that could lead to unauthorized database access, data breaches, and compromise of connected systems.

Recommendation

Remove all credential logging immediately and implement secret masking in logs using structured logging libraries that automatically redact sensitive values.

Status

Resolved

[M-02] defi-frontend - Hardcoded Authorization Bearer token

Description

In line 254 of `src/Services/Utils.ts`, an Authorization Bearer token is hardcoded for the `GALA_ANALYTIC_TOKEN` to log events for analytics usage.

Impact

If an attacker gains access to the source code, they can potentially reuse the token to gain unauthorized or privileged access to protected APIs or internal systems.

Recommendation

Consider storing the token as an environmental variable.

Status

Acknowledged

[M-03] bundling-backend - Missing rate limiting

Description

None of the controllers or middleware enforces any throttling, leaving all routes wide open to high-volume requests.

Impact

Attackers can flood the API with requests, exhausting CPU, memory, or database connections and causing a denial of service.

Recommendation

Integrate a rate-limiting solution to cap request frequency per IP or API key.

Status

Acknowledged

[M-04] bundling-backend - Outbound HTTP requests lack timeouts

Description

The BundlerService's HTTP calls to the external blockchain API do not specify a timeout, allowing requests to hang indefinitely under adverse conditions. In `src/api/bundler/bundler.service.ts` the `sendBatchRequest` method invokes:

```
return firstValueFrom(  
  httpService.post(url, batch, { headers: { 'Content-Type': 'application/json' } })  
);
```

with no `timeout` option. If the upstream API stalls, the cron job remains blocked and resources are tied up.

Impactx

Unresponsive external dependencies can exhaust the event loop and lead to application hangs or cascading failures.

Recommendation

Configure a reasonable timeout (e.g., 10 seconds) on all outbound HTTP requests.

Status

Resolved

[M-05] defi-backend - Missing blockchain transaction confirmation verification

Description

After submitting a transaction request to the chain API, the code treats any `Status` flag as success without waiting for on-chain confirmation or checking for reorgs. In `src/helper/user.helper.ts`:

```
const res = await firstValueFrom(httpService.post(url, transfer));  
  
if (!res.data.Status) throw new Error('Error in transferring faucets');  
  
else return true;
```

No subsequent polling of transaction receipt or confirmation depth is performed.

Impact

The application may report success even if the transaction is later dropped or reverted on the blockchain, leading to inconsistencies.

Recommendation

Fetch the transaction hash from the response, wait for a configurable number of block confirmations, and verify the receipt status before reporting success.

Status

Acknowledged

[M-06] defi-frontend, defi-backend, bundling-backend - TransferDexPosition
is not implemented in the user interface

Description

In the Dex chaincode, there is a `TransferDexPosition` functionality that allows users to transfer their Dex position to a recipient. However, the frontend and backend do not implement entry points to support this functionality.

Impact

Users are unable to use the `TransferDexPosition` functionality as expected.

Recommendation

Consider implementing the `TransferDexPosition` functionality in the user interface.

Status

Acknowledged

[M-07] dex - Negligible positions are not deleted

Description

The `removePositionIfEmpty` function removes a user's `DexPositionData` if they have negligible liquidity and tokens owned. This is done by calling `deleteChainObject(ctx, position)` in line 46 of `src/chaincode/dex/removePositionIfEmpty.ts`.

However, the `burn` and `collect` functions restore it by calling `putChainObject(ctx, position)` in lines 176 of `src/chaincode/dex/burn.ts` and 122 of `src/chaincode/dex/collect.ts`.

This means the user position will not be removed, as the `burn` and `collect` functions store it back.

Impact

The expected `DexPositionData` is not removed as intended.

Recommendation

Consider only calling `putChainObject(ctx, position)` if there is no need to remove the user's old position.

Status

Resolved

[M-08] dex - setProtocolFee can be set to negative or over-bounded values**Description**

In line 42 of `src/chaincode/dex/setProtocolFee.ts`, the authority can configure the protocol fee. However, no validation enforces the protocol fee to be within the range of 0 to 100%.

It is worth mentioning that a `configureProtocolFee` function in line 367 of `src/api/types/DexV3Pool.ts` performs the range validation, but the function is not implemented.

Impact

Protocol fees can be configured to invalid values, preventing new pool creations.

Recommendation

Consider implementing the `configureProtocolFee` functionality when setting the protocol fee.

Status

Resolved

[M-09] bundling-backend - Unauthenticated bundle endpoint allows mass transaction injection

Description

The `POST /bundle` endpoint in the `BundlerController` accepts arbitrary payloads without any authentication or authorization checks. An attacker can submit any `signedDto` object, and the service will write it to the database and later include it in on-chain batch requests.

Vulnerability Details

In `src/api/bundler/bundler.controller.ts`, the `bundleStrings` method is exposed publicly.

```
@Post()
async bundleStrings(@Body() payload: TransactionMethodDto) {
  const response = await this.bundlerService.handleTransactions(payload);
  ...
  return response;
}
```

The `BundlerService` stores the `payload.signedDto` directly and, when the cron job triggers, constructs batch operations from it:

```
const operations = dtos.map(dto => ({
  dto: dto.signedDto as ChainCallDTO,
  method: dto.method,
  uniqueId: dto.uniqueId,
}));
await httpService.post(url, batchDto);
```

Because there is no client identity or any rate-limit implemented, anyone can inject arbitrary transaction payloads that the backend will forward to the blockchain API.



Proof of Concept:

Example request:

```
POST /bundle HTTP/2

Host: bundle-backend-test3.defi.gala.com

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

Accept: application/json

Content-Type: application/json

Content-Length: 744


{
  "method": "AddLiquidity",
  "stringsInstructions": [
    "pool$attacker$pool",
    "tokenBalance$attacker$pool"
  ],
  "signedDto": {
    "owner": "0xDEADBEEFDEADBEEFDEADBEEFDEADBEEFDEADBEEF",
    "token0": {
      "type": "Unit",
      "category": "Unit",
      "collection": "GALA",
      "additionalKey": "none"
    },
    "token1": {
      "type": "Unit",
```

```

    "category": "Unit",

    "collection": "SILK",

    "additionalKey": "none"

  },

  "fee": "3000",

  "tickLower": "0",

  "tickUpper": "0",

  "amount0Desired": "0.1",

  "amount1Desired": "0.1",

  "amount0Min": "0",

  "amount1Min": "0",

  "signature": "0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"

}

}

```

Example response:

```

HTTP/2 201 Created

Date: Sun, 22 Jun 2025 09:53:43 GMT

Content-Type: application/json; charset=utf-8

Content-Length: 94

Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https:
data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src
'none';script-src 'self';script-src-attr 'none';style-src 'self' https:
'unsafe-inline';upgrade-insecure-requests

Cross-Origin-Opener-Policy: same-origin

Cross-Origin-Resource-Policy: same-origin

Origin-Agent-Cluster: ?1

Referrer-Policy: same-origin

```



```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

```
X-Content-Type-Options: nosniff
```

```
X-Dns-Prefetch-Control: off
```

```
X-Download-Options: noopen
```

```
X-Frame-Options: SAMEORIGIN
```

```
X-Permitted-Cross-Domain-Policies: none
```

```
X-Xss-Protection: 0
```

```
Access-Control-Allow-Origin: *
```

```
Access-Control-Allow-Credentials: true
```

```
Etag: W/"5e-2kwh+ef4ygNLqltqR+UZLfXjhAU"
```

```
{"data":"ade8a72c-46b8-44e7-a620-be02049a6c69","message":"Transaction  
Received","error":false}
```

Impact

An attacker can spam unauthorized or malicious transactions to the blockchain, leading to a denial of service or unintended fund movements.

Recommendation

Require authentication or API-key validation on `/bundle` so only trusted clients may submit transactions.

Status

Acknowledged

Low

[L-01] oracle-lambda - Verbose error logging

Description

The Lambda function logs complete error objects including stack traces throughout the codebase, which could expose sensitive information about the application's internal structure, database schema, and implementation details to anyone with access to CloudWatch logs.

The error handling pattern used throughout the application logs full error objects that contain detailed stack traces and internal state information:

```
} catch (err) {  
    console.error("Error in connection to MySQL", err.message);  
    throw err; // re-throw so the caller can handle it too  
}  
  
} catch (err) {  
    console.error("Error fetching active tokens:", err.message);  
    console.error("Full error:", err);  
    throw err;  
}
```

Attackers with log access can gather detailed information about the application's internals, database structure, and potential vulnerabilities.

Recommendation

Implement structured logging that sanitizes error messages in production environments and logs only essential information needed for debugging.

Status

Resolved

[L-02] defi-backend - Unvalidated external price data from on-chain pools

Description

Price quotes retrieved from external pool endpoints are accepted without sanity checks for absurd or manipulated values.

Vulnerability Details

In `src/trade/trade.service.ts`, inside `getPriceOfTokenFromPool`:

```
const response = await httpService.post(`${this.baseUrl}/${QUOTE_EXACT_AMOUNT}`,  
quoteChainDto);  
  
const quoteAmount = new BigNumber(response.data.Data.amount0);  
  
const resultAmount = new BigNumber(response.data.Data.amount1);  
  
return { data: resultAmount.abs() };
```

No range checks or fallback to a secondary source is implemented.

Impact

A malicious or compromised node could return extreme price values, causing trades to execute at unfair rates or triggering liquidations.

Recommendation

Implement deviation bounds and cross-verify with multiple independent price oracles before accepting a price quote.

Status

Resolved

[L-03] oracle-lambda - Usage of `conn.query` may expose SQL injection risks**Description**

In lines 114, 226, and 290 of the `index.js` file, `conn.query` is used to store updates into the database. This is problematic because SQL injection is possible as prepared statements are not automatically utilized.

Impact

SQL injection may occur if the input is controlled by malicious attackers.

Recommendation

Consider using `conn.execute` to perform the updates to the database.

Status

Resolved

[L-04] defi-frontend - The Deadline functionality is commented out**Description**

In line 204 of `src/components/modals/settingModal/SettingModal.tsx`, the deadline functionality is commented out.

Impact

Users will not be able to specify a deadline for the swap DTO to be executed. For example, users may choose to perform a swap with a tolerably higher slippage during unfavorable market conditions. Missing deadline implementations allow the swap to be executed when the market recovers, potentially costing the user funds.

Recommendation

Consider implementing a deadline.

Status

Acknowledged

[L-05] bundling-backend - Health check endpoint always returns success

Description

The `/health` and `/health/debug-sentry` endpoints are hard-coded to return a result without actually verifying the status of the database, external APIs, or other dependencies:

```
@Controller('health')

export class HealthController {

  constructor(

    private readonly health: HealthCheckService,

    private readonly http: HttpHealthIndicator,

    private readonly config: ConfigService,

    @Inject(databaseProvider[0].provide)

    private readonly sequelize: Sequelize,

  ) {}

  @Get()

  @HealthCheck()

  public checkHealth() {

    return {

      status: 200,

      message: 'App is running',

    };

  }

  @Get('/debug-sentry')
```

```
public getError() {  
    throw new Error('Sentry is throwing error');  
}  
  
}
```

Due to this, monitoring systems receive false-positive health signals and may fail to detect real service outages.

Recommendation

Use `HealthCheckService.check()` with proper indicators (database, HTTP, etc.) to perform genuine health validations.

Status

Resolved

[L-06] defi-backend - Broken onboarding timestamp validation

Description

The timestamp check intended to limit the validity of onboarding messages always passes because the configured time window is undefined. In `src/onboarding/onboarding.service.ts`:

```
const msgTimestamp = Number(data.message.split(' ').pop());  
  
if (msgTimestamp > Date.now() || msgTimestamp < Date.now() -  
this.config.get('time.authMsgValidationTime')) {  
  throw new CustomError('Invalid message', 400);  
}
```

`time.authMsgValidationTime` is never set in configuration, so the second condition compares to NaN and always evaluates false. Hence, attackers can reuse old signed messages indefinitely to register new accounts, defeating replay protection.

Recommendation

Define a concrete message validity interval (for example, 5 minutes) and enforce it by reading a known config property or using a default.

Status

Resolved

[L-07] defi-frontend - Dead placeholder auth token sent on every request**Description**

The frontend initializes and continues to send a hard-coded `"AUTH_TOKEN"` header on all API calls, even when the user is unauthenticated. In `src/Redux/Slices/user.slice.ts`, the slice's `initialState` sets

```
authToken: "AUTH_TOKEN",
```

Due to this, clients use a meaningless token.

Recommendation

Initialize `authToken` to an empty string and only set it after a successful login. Refactor API helpers to omit auth headers when no valid token is present.

Status

Resolved

[L-08] defi-frontend - Exposure of internal environment endpoints & admin wallet address in bundle

Description

Compiled frontend code embeds URLs for `dev/stage/qa/prod` backends and an admin wallet literal, all visible in the shipped JS. In `Services/Utils.ts`, the `switch(ENVIRONMENT)` block defines dozens of hostnames (e.g. `dex-backend-dev1.defi.gala.com`, `bundle-backend-qa1.defi.gala.com`, etc.) and assigns `ADMIN_WALLET_ADDRESS`.

After building, these values appear in `assets/index-By5IMFz7.js` and can be extracted by anyone. Due to this, attackers learn internal infrastructure topology and a privileged wallet address, aiding targeted attacks or phishing.

Recommendation

Load environment-specific URLs and sensitive addresses from a secure backend at runtime or via server-only config. Strip unused environment cases from the production bundle.

Status

Acknowledged

[L-09] defi-backend - Missing validation of special characters

Description

Errors displayed by the application contain unvalidated special characters (e.g. HTML). However, the content type is typically JSON, which prevents potential XSS attacks, along with CSP. However it is still a good practice to sanitize any HTML characters. Example of such an error is presented below:

```
GET
/explorer/pools?limit=10&page=1&search=frffff&hb68e%3cscript%3ealert(1)%3c%2fscript%3ent9j7=1 HTTP/2

Host: dex-backend-test3.defi.gala.com

X-Auth-Token: AUTH_TOKEN

Sec-Ch-Ua-Platform: "Windows"

Accept-Language: en-US,en;q=0.9

Accept: application/json, text/plain, */*

Sec-Ch-Ua: "Chromium";v="137", "Not/A)Brand";v="24"

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36

Sec-Ch-Ua-Mobile: ?0

Origin: https://dex-frontend-test3.defi.gala.com

Sec-Fetch-Site: same-site

Sec-Fetch-Mode: cors

Sec-Fetch-Dest: empty

Referer: https://dex-frontend-test3.defi.gala.com/

Accept-Encoding: gzip, deflate, br

Priority: u=1, i
```

Response:

```

HTTP/2 400 Bad Request

Date: Sun, 22 Jun 2025 15:28:38 GMT

Content-Type: application/json; charset=utf-8

Content-Length: 101

Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https:
data::form-action 'self';frame-ancestors 'self';img-src 'self' data::object-src
'none';script-src 'self';script-src-attr 'none';style-src 'self' https:
'unsafe-inline';upgrade-insecure-requests

Cross-Origin-Opener-Policy: same-origin

Cross-Origin-Resource-Policy: same-origin

Origin-Agent-Cluster: ?1

Referrer-Policy: same-origin

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

X-Content-Type-Options: nosniff

X-Dns-Prefetch-Control: off

X-Download-Options: noopen

X-Frame-Options: SAMEORIGIN

X-Permitted-Cross-Domain-Policies: none

X-Xss-Protection: 0

Access-Control-Allow-Origin: *

Access-Control-Allow-Credentials: true

Etag: W/"65-maCgEyfHyPEilNRbozzeIkEn8"

Vary: Accept-Encoding


{"status":400,"error":true,"message":"property      hb68e<script>alert(1)</script>nt9j7
should not exist"}

```

As mentioned, the response contains unsanitized special characters.



Recommendation

Strip/HTML Encode any special characters. Avoid printing user input as part of errors and generally emitting backend errors directly to users; Produce a friendly error message instead and only log full error content to the backend logs that are accessible only to the administrators.

Status

Resolved

[L-10] dex - TransferDexPosition is not implemented in FeeGateCodes

Description

In line 19 of `src/chaincode/dexLaunchpadFeeGate.ts`, the `FeeGateCodes_enum` does not implement `TransferDexPosition`, which is a functionality that allows users to transfer positions.

Impact

Gas fees may not be charged sufficiently for the `TransferDexPosition` functionality.

Recommendation

Consider implementing the fee system for `TransferDexPosition`.

Status

Resolved

[L-11] dex - GetDexFeeConfiguration cannot be queried by anyone**Description**

In line 246 of `src/chaincode/DexV3Contract.ts`, queries are expected to be used by users to retrieve the Dex configuration. However, there exist restrictions that ensure only the "CuratorOrg" organization can access them, as seen in line 244 of `src/chaincode/DexV3Contract.ts` and lines 141 and 149-151 of `src/chaincode/dex/getFunctions.ts`.

Impact

Users and third-party chaincode cannot query the Dex configuration.

Recommendation

Consider removing the organization requirement.

Status

Acknowledged

QA

[Q-01] defi-backend - Incorrect sortBy parameter error message

Description

In line 5 of `src/api/explore/dto/explore.dto.ts`, the `sortBy` parameter only accepts enum fields of `tv1` or `volume30d`. If the user specifies any other values, the error message will indicate that “sortBy can only be either tv1 or volume_30d”.

This is incorrect because it should correctly indicate `volume30d` instead of `volume_30d`.

Recommendation

Consider updating the error message to be “sortBy can only be either tv1 or volume30d”.

Status

Resolved

[Q-02] defi-backend - Unused price variable

Description

In line 672 of `src/api/trade/trade.service.ts`, the `price` variable is set to `priceRes.data`. However, the variable is unused throughout the execution.

Recommendation

Consider setting the `data` parameter in line 678 to use `price.toString()`, similar to line 667.

Status

Resolved

[Q-03] defi-backend - Private key exposure risk

Description

The code directly loads the blockchain private key into application memory and uses it for signing, exposing it if environment variables or logs are compromised. In `src/helper/user.helper.ts`, the `transferFunds` function performs:

```
transfer.sign(config.get('web3.privateKey'));
```

The private key is read from plain text environment variables and used directly in application code.

Recommendation

If an additional security measure is needed, use a hardware security module (HSM) or managed key service to perform signing and avoid storing raw private keys in environment variables.

Status

Acknowledged

[Q-04] dex - SwapDto constructor ignores amountInMaximum and amountOutMinimum

Description

In line 204 of `src/api/types/DexDtos.ts`, the `SwapDto` constructor ignores the `amountInMaximum` and `amountOutMinimum` fields. This means that if a call to `new SwapDto()` is performed, the `amountInMaximum` and `amountOutMinimum` fields will not be included.

Recommendation

Consider adding the `amountInMaximum` and `amountOutMinimum` fields to the constructor.

Status

Resolved

[Q-05] dex - Incorrect `SlippageToleranceExceededError` **error****Description**

In line 131 of `src/chaincode/dex/swap.ts`, the `${dto.amountInMaximum}` error used is incorrect. It should be substituted with `amountOutMinimum`, and the error string should reflect “minimum received tokens is **more** than the actual received amount”.

Recommendation

Consider replacing the error message with `amountOutMinimum` and using “more than the actual received amount”.

Status

Resolved

Conclusion

After Hashlock's analysis, the Gala project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.